

DS-1000: A Benchmark for Data Science Code That Models Can't Just Memorize

ICML 2023: HKU, Meta AI, and collaborators build 1,000 real StackOverflow problems with reliable execution-based evaluation

When we measure code-generation models, we reach for benchmarks like HumanEval and APPS: algorithmic puzzles with clean, self-contained prompts. They test whether a model can reason about algorithms, but they look very little like the coding most people actually do, and almost nothing like day-to-day data science. A real data science task is rarely "implement quicksort"; it's closer to "I have this DataFrame and want the inverse of each column added with a matching prefix, the pythonic way."

These are exactly the problems where a model could help most. Data science runs on specialized libraries (NumPy, Pandas, PyTorch), and simply remembering their sprawling APIs is a real barrier for everyday users. Code-generation models could lower it, but the community lacked a benchmark that measures genuine progress. Most existing benchmarks test competition-style algorithms or score answers with surface-form metrics like BLEU, which drift from what a programmer intends as models get stronger: text that looks right scores well even when the code doesn't run.

This ICML 2023 paper introduces DS-1000, a natural and reliable benchmark for data science code generation. It collects 1,000 real problems from StackOverflow across seven widely used Python libraries; it judges answers by executing code rather than comparing text; and it proactively perturbs problems so a model can't pass by recalling a memorized answer. Together, those three ingredients give the field a trustworthy yardstick that is genuinely hard to game.

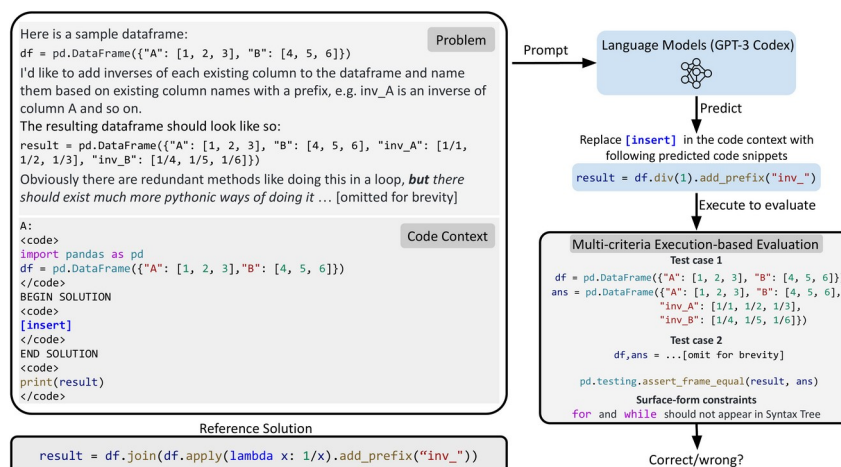


Figure 1. An example DS-1000 problem. The model fills code into the [insert] slot on the left; the completed program is then executed against multi-criteria evaluation, which combines functional test cases with surface-form constraints, with a reference solution shown at the bottom left.

Two problems: the tasks aren't real, and the ruler isn't accurate

Real data science questions almost never arrive as tidy textbook prompts. On StackOverflow, people ask with a lot of context attached: they paste the broken code they wrote, the error message they hit, and concrete input-output examples of what they actually want. That context is precisely what a model should learn to use, yet prior benchmarks strip it away, leaving only an abstract restatement of the task.

The other half of the problem is the evaluation itself. When models were weak, a surface-form metric like BLEU was a passable proxy. But as models improve, the gap between "looks similar to the reference" and "actually runs correctly" only widens: a snippet that closely mimics the reference answer but is logically wrong still scores well. Measuring real progress requires switching to reliable, execution-based evaluation that runs the code against test cases. DS-1000 targets both failures at once: the problems should be natural and carry real context, and the evaluation should be execution-based and reliable.

How it's built: a five-stage pipeline

Turning "wild" StackOverflow questions into problems that can be graded automatically and can't be gamed by memorization is not easy. DS-1000 does it with a five-stage pipeline, and every problem, solution, and metric is cross-reviewed by at least three expert annotators.

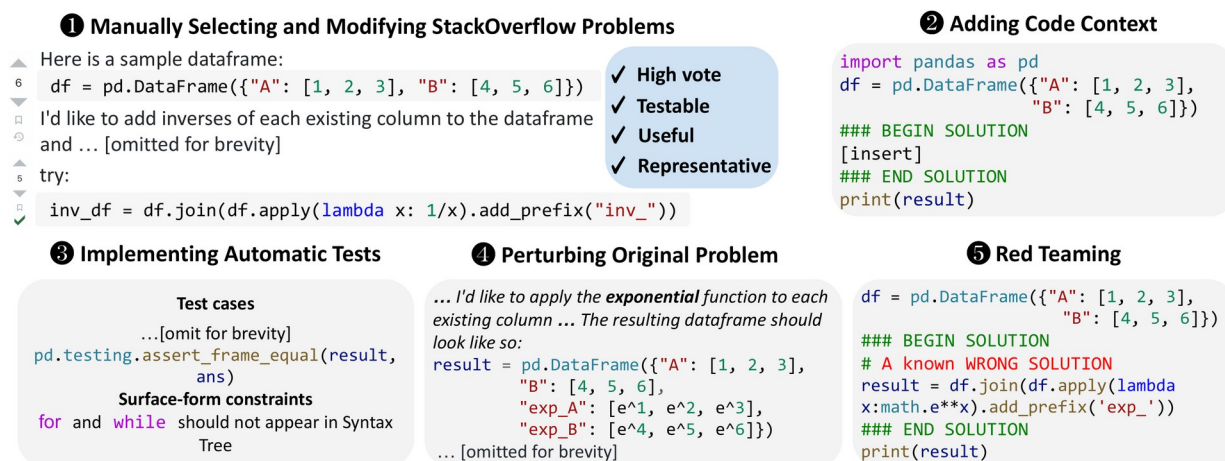


Figure 2. The five-stage pipeline for building DS-1000: (1) manually select and rewrite high-quality StackOverflow problems, (2) add code context with insertion markers, (3) implement automatic tests (test cases plus surface-form constraints), (4) perturb the original problem to resist memorization, and (5) red-team the evaluation.

The first stage selects and rewrites problems: annotators keep posts that are high-vote, testable, useful, and representative, then rewrite the colloquial original for clarity. The second stage adds a code context with an insertion marker that tells the model exactly where to write (the [insert] slot from Figure 1). The third stage implements automatic tests: it runs test cases to check functional correctness and also enforces surface-form constraints on the syntax tree,

such as forbidding keywords like `for/while` or specific APIs. The first two stages keep the problems natural; the third makes grading reliable.

The clever part is stage four: perturbation. A model trained on much of the web has likely already seen, and possibly memorized, the original StackOverflow answers. So the authors rewrite each original problem. Surface perturbations change only the wording and leave the reference solution unchanged; semantic perturbations change the intent, so the reference solution changes too; and a batch of deliberately difficult rewrites raises the bar further. Stage five is red-teaming: the evaluation script must reject known-wrong solutions to be accepted. The result is a benchmark that stays close to reality while being hard to pass from memory.

What the data looks like: 1,000 problems with thicker context

DS-1000 ends up with 1,000 problems drawn from 451 unique StackOverflow posts, spanning seven libraries: NumPy, Pandas, TensorFlow, PyTorch, SciPy, Scikit-learn, and Matplotlib. More than half of the problems are modified from their sources to resist memorization: specifically 152 surface perturbations, 235 semantic perturbations, and 162 difficult rewrites.

Statistically, each problem carries on average 1.6 test cases, a prompt of about 140 words, and a 3.6-line reference solution, and 19.4% of problems come with surface-form constraints. That 140-word average is far richer than comparable data science datasets: DSP averages 71.9 words and CoNaLa just 13.8. Longer prompts preserve more of the real, messy context that DS-1000 keeps on purpose and that most other benchmarks quietly discard along the way.

How reliable is the evaluation: accepted answers are almost always right

A benchmark's reliability ultimately comes down to one question: when it says a solution is correct, is it really? The authors quantified this with human review. Among all solutions the benchmark accepts, only 1.8% are actually wrong (the sample-level false discovery rate); among those it rejects, only 0.5% are actually correct (the false omission rate). Put more concretely, DS-1000's problem-level false-positive rate is 5.7%, compared with the 60% reported for the same metric on APPS. Execution-based, multi-criteria evaluation pushes misjudgment down to a very low level.

How the models do: the best reaches only 43.3%

The authors evaluate five pre-trained code models from three families, and DS-1000 separates them clearly. The strongest is Codex-002 in the Insertion (infilling) format, and even it averages only 43.3%, nontrivial but far from solved, leaving substantial room to improve. Across the five models, accuracy ranges from 7.4% up to 43.3%. Notably, the Insertion format supplies right-hand context, and that alone gives Codex-002 a 4.1% average gain over the Completion format, underscoring that infilling genuinely helps on data science tasks. Weaker models such as CodeGen-6B and InCoder-6B fall below 5% on some libraries.

Why perturb: a probe that exposes memorization

Is stage-four perturbation really necessary? The authors ran a revealing probe on the widely circulated numpy-100 problem set. Codex-002 scores 72.5% on the original numpy-100, but once the problems are perturbed its accuracy collapses to 40.6%, and in 36% of the semantic-perturbation cases the model still emits the answer to the original problem verbatim, even though that answer is now wrong. That is the signature of recalling a memorized solution rather than reasoning.

Table 1. Accuracy before and after perturbation (Codex-002). A heavily copied set like numpy-100 collapses under perturbation, whereas DS-1000, which is repeated far less online, degrades only gently.

Setting	Original	Perturbed	Surface drop	Semantic drop
numpy-100	72.5%	40.6%	-50.8%	-23.6%
DS-1000	—	—	-3.4%	-9.0%

By contrast, the same perturbation on DS-1000 causes a much gentler drop (about 3.4% for surface and 9.0% for semantic), because these problems appear less often online and leave less room to memorize. The two ends of the comparison make the same point: memorization is a real risk, and deliberate perturbation is a simple, effective defense that keeps the benchmark challenging even for stronger future models.

Takeaway: a realistic, reliable, memorization-resistant ruler

DS-1000 gives data science code generation a rare kind of yardstick: its problems come from real StackOverflow questions, so they stay close to practice; its evaluation runs the code and layers on surface-form constraints, so its scores are trustworthy and rarely misjudged; and its deliberate perturbation blocks memorization, so it is hard to game and stays meaningful as models improve. The fact that even the strongest model reaches only 43.3% is itself a clear marker of how much headroom remains. For anyone working on data science code generation, DS-1000 is both an honest measuring tool and a to-do list that is far from finished.